Multi-Tenant Healthcare Microservices Platform

A cloud-native, multi-tenant healthcare SaaS platform built using .NET microservices, deployed on AWS ECS Fargate, with Terraform IaC, event-driven architecture (SNS/SQS), a globally distributed frontend hosted on Amazon S3 + CloudFront. and a multi-pipeline CI/CD system supporting blue/green and rolling deployments.

System Overview

This platform is designed for scalable healthcare workloads with:

- Multi-tenant isolation
- Event-driven microservices
- Secure file/document management
- Independent deployment pipelines
- Fully automated infrastructure provisioning

High-Level Architecture

Frontend Layer

- React JS, Typescript, Material UI SPA
- Hosted in Amazon S3
- Delivered globally through Amazon CloudFront
- Static asset caching
- Low-latency content delivery

Core AWS foundation:

- Amazon VPC — Network isolation
- Public Subnets — ALB + NAT Gateway
- Private Subnets — ECS workloads
- DB Subnets — Isolated database layer
- Amazon ECS (Fargate) — Runs all microservices
- Application Load Balancer — Entry point for gateway API
- Amazon RDS (SQL Server) — Persistent data storage
- Amazon S3 — File storage layer
- NAT Gateway — outbound internet access
- VPC Endpoints — private AWS service communication
- Secret Manager - store platform secrets such as db and microservices secrets

Frontend Architecture

The Single Page Application (SPA) is deployed as static assets.

Hosting

- Frontend build artifacts are uploaded to:
  - Amazon S3
- Content is distributed through:
  - Amazon CloudFront

Benefits

- Global edge caching

- Reduced latency
- Reduced backend load
- Highly available frontend
- Cost-effective hosting
- Automatic HTTPS

Microservices Architecture

Public/API Layer

- gateway-api -> Single entry point (routing, auth enforcement)

Domain Services

- auth-api -> Authentication & JWT issuance using AWS cognito user pool
- tenant-api -> Tenant provisioning & management
- patient-api -> Patient records
- appointment-api -> Scheduling system
- billing-api -> Invoicing & payments
- notification-api -> Communication service
- doctor-api -> Doctor records
- file-service -> Document upload/download via S3
- audit-api -> logging

Background Processing

- worker-service -> Async processing via SQS (notifications, billing, audits etc...)

Multi-Tenancy Model

- Tenant isolation via TenantId in every request
- Middleware-based tenant resolution via TenantMiddleware
- Shared database with:
  - Row-level isolation (default)
  - Optional schema/database-per-tenant (enterprise mode) -> for later implementation

Request Flow

Client → Application Load Balancer
- ALB → gateway-api
- Gateway-api → internal microservices (private VPC networking)
- Services interact with:
  - Amazon RDS (SQL SERVER)
  - Amazon S3
- Response returns via gateway

Event-Driven Architecture

The system uses asynchronous messaging for scalability and decoupling.

Messaging Backbone
- Amazon SNS — event publisher (fan-out hub)
- Amazon SQS — durable message queues

- Worker services consume events independently

SNS → SQS Fan-Out Pattern

- Service publishes event → SNS
- SNS fans out to multiple SQS queues
- Worker-service processes queues asynchronously

Key Events

- PatientCreated
- AppointmentScheduled
- BillingGenerated
- FileUploaded
- TenantProvisioned
- etc...

Worker-Service Responsibilities

- Notification processing (email/SMS) -> future implementation
- Billing calculations future implementation
- Audit logging
- Event enrichment
- Background jobs

File-Service (S3 Document System)

- Storage Layer
  - Amazon S3

Responsibilities

- Upload medical documents
- Download files securely
- Generate pre-signed URLs
- Maintain file metadata per tenant -> Future impl

File Upload Flow

- Client uploads file → file-service
- Service validates tenant + permissions
- File stored in S3
- Metadata stored in DB
- log event → audit log (FileUploaded)

File Download Flow

- Client requests file
- Access validated by file-service
- Pre-signed S3 URL generated
- Direct download from S3 (no backend load)

Deployment Architecture

All services run in:

- Amazon ECS (serverless containers)

Networking Layout

- Public Subnets → ALB, NAT Gateway
- Private Subnets → ECS services

- DB Subnets → RDS only

CI/CD Architecture (Multi-Pipeline Strategy)

This system uses 4 independent CI/CD pipelines:

1. Gateway API Pipeline (Blue/Green Deployment)
   - Deploys gateway-api
   - Uses ECS Blue/Green deployment
   - Zero-downtime release strategy
   - Automated rollback on failure
2. Internal Microservices Pipeline (Rolling Deployment)
   - Deploys:
     - patient-api
     - billing-api
     - appointment-api
     - tenant-api
     - doctor-api
     - auth-api
     - audit-api
     - notification-api
     - file-service
   - Uses ECS rolling updates
   - Faster deployments for internal services
3. Worker-Service Pipeline
   - Builds and deploys background worker service
   - Consumes SQS queues

- Independent scaling from APIs
- Event-driven deployment lifecycle

4. Terraform Infrastructure Pipeline
   - Manages all infrastructure state
   - Applies:
     - VPC
     - ECS clusters
     - ALB
     - RDS
     - S3 buckets
     - SNS/SQS
     - IAM roles
   - Uses remote state backend (S3 + uselock)

Security Model

- Private ECS services (no public exposure)
- IAM roles per service
- Security Groups restrict service-to-service traffic
- Secrets stored in AWS Secrets Manager
- TLS termination at ALB
- S3 access controlled via IAM policies

Observability

- Logs → Amazon CloudWatch Logs
- Metrics → CloudWatch Metrics
- Health checks → ALB target groups

Tech Stack

- .NET 8 (ASP.NET Core Microservices)
- Docker
- Terraform
- AWS ECS Fargate
- AWS RDS (SQL Server)
- AWS S3
- AWS Cloudfront
- AWS SNS/SQS
- JWT Authentication
- REST APIs
- CI/CD (GitHub Actions / AWS CodePipeline)

Author

- A cloud-native healthcare SaaS platform designed for scalability, resilience, and enterprise-grade multi-tenancy using AWS serverless container architecture.